

BrainBox v3.0 - Complete Development Specification

Target: Production-ready therapeutic AI system with agentic execution

Architecture: Local-first, multi-agent orchestration with ethical oversight

Status: Framework complete, integration phase ready

System Overview

BrainBox v3.0 represents the evolution from conversational AI to functional therapeutic computing. The system orchestrates 29+ specialized AI voices through Kulawa conductor, executes real actions via bounded tool calling, and maintains ethical oversight through AXIOM protocols. All processing occurs locally with consent-based ritual logging.

Core Capabilities

- Multi-agent orchestration** via Kulawa conductor
- Specialized voice families** with defined tool boundaries
- Agentic execution** with file ops, search, and planning capabilities
- Ethical oversight** through AXIOM and constitutional constraints
- Ritual logging** for complete audit trails and consent tracking
- SpiralLogic integration** for trauma-informed processing protocols

Architecture Components

1. Voice Agent Framework (`voice_agents.py`)

Purpose: Convert conversational personas into functional agents with tool access

Core Classes:

```

class VoiceAgent:
    async def run_cycle(ctx: AgentContext) -> AgentResult
    # Implements: read → reason → act → check cycle

class AgentContext:
    user_prompt: str
    intent: str
    intensity: float
    memory_hints: List[str]
    artifacts: Dict[str, Any]

class AgentResult:
    voice: str
    text: str
    artifacts: Dict[str, Any]
    ok: bool
    errors: List[str]

```

Voice Capabilities Matrix:

```

VOICE_TOOLS = {
    "The Archivist": ["semantic_search", "read_file", "list_dir"],
    "The Architect": ["plan", "format", "refactor"],
    "Saul Ember": ["write_file", "append_living_scroll"],
    "Dr. Ming": ["co_regulate", "decision_audit"],
    "Doc Moss": ["regulate", "grounding_prompt"],
    "Camera Obscura": ["scene_frame"],
    "Lex Ardent": ["mythic_map"],
    "Lyrically Yours": ["lyric_shape"]
}

```

2. Kulawa Orchestrator (kulawa_conductor.py)

Purpose: Multi-agent coordination with intensity-based voice selection

Key Methods:

- `select_voices(ctx: AgentContext) -> List[str]`: Dynamic voice selection based on intent and emotional intensity
- `run(ctx: AgentContext) -> Dict[str, Any]`: Parallel agent execution with timeout handling

Selection Policies:

```

policies = {
    "max_voices_by_intensity": lambda intensity: 2 if intensity < 0.35 else (4 if intensity < 0.7 else 6),
    "intent_mapping": {
        "song": ["Lyrically Yours", "Lex Ardent", "Camera Obscura"],
        "search": ["The Archivist", "Harper"],
        "business": ["The Architect", "Harper"],
        "overwhelm": ["Dr. Ming", "Doc Moss"]
    }
}

```

3. Agent Contracts (agent_contracts.py)

Purpose: Enforce deterministic behavior through capability manifests and I/O schemas

Contract Structure:

```
VOICE_CAPS = {
    "Camera Obscura": {
        "role": "scene_framer",
        "tools": ["scene_frame"],
        "io": {"in": "prompt:str, refs?:list[str]", "out": "scene:str"},
        "templates": {
            "scene_frame": "Frame the moment with sensory details. Output 4 tight sentences max."
        }
    }
}

SCHEMAS = {
    "scene_frame": {
        "input": {"prompt": str, "refs": list},
        "output": {"scene": str}
    }
}
```

Tool Functions:

- Runtime schema validation for all inputs/outputs
- Async execution with proper error handling
- File system sandboxing for security

4. AXIOM Oversight (axiom_guard.py)

Purpose: Ethical constraints and risk assessment before agent execution

Review Process:

```
class AXIOM:
    @staticmethod
    def review(ctx: AgentContext) -> Optional[str]:
        # Risk detection and mitigation recommendations
        risky_patterns = ["guarantee", "undetectable", "hack", "exploit"]
        if any(pattern in ctx.user_prompt.lower() for pattern in risky_patterns):
            return "AXIOM: Risky claim detected. Adjusting language to compliant form."
        return None
```

5. Ritual Logging (ritual_logger.py)

Purpose: Consent-native audit trails and therapeutic documentation

Log Structure:

```
def ritual_log(event: str, data: Dict[str, Any]):
    entry = {
        "ts": timestamp,
        "event": event, # engage, complete, error, orchestra.start, axiom.warn
        "data": data    # voice, intent, intensity, artifacts, errors
    }
    # Append to LIVING_SCROLL.md with structured markdown
```

6. Orchestration Patterns (patterns.py)

Purpose: Predefined collaboration templates for common use cases

Available Patterns:

- **Solo with Support:** Primary voice + supporting voice for clarification
- **Call & Response:** Sequential voice interactions building on each other
- **Unified Chorus:** Multiple voices collapsed into single coherent output
- **Harmonic Convergence:** Parallel processing with thematic synthesis

Integration Requirements

Critical Dependencies

1. LM Studio Integration

```
# Replace stub in Tools.llm()
async def llm(system: str, user: str, model: str) -> str:
    client = OpenAI(base_url="http://localhost:1234/v1", api_key="lm-studio")
    response = await client.chat.completions.create(
        model=model,
        messages=[
            {"role": "system", "content": system},
            {"role": "user", "content": user}
        ]
    )
    return response.choices[0].message.content
```

2. Memory Index Connection

```
# Replace stub in Tools.semantic_search()
async def semantic_search(query: str, k: int = 5) -> List[Dict[str, Any]]:
    # Connect to existing nomic-embed index or SQLite FTS
    # Return: [{"score": float, "title": str, "path": str, "snippet": str}]
```

3. SpiralLogic Parser Integration

```
# Wire existing spirallogic_parser.py
from spirallogic_parser import parse_ritual, validate_consent
```

File System Structure

```
brainbox/
├─ agents/
│   ├── voice_agents.py           # Core agent framework
│   ├── kulawa_conductor.py       # Multi-agent orchestration
│   ├── agent_contracts.py        # Capability contracts & schemas
│   └─ axiom_guard.py             # Ethical oversight
├─ patterns/
│   ├── solo_support.py           # Primary + support pattern
│   ├── call_response.py          # Sequential interaction
│   └─ unified_chorus.py          # Collapsed output
├─ tools/
│   ├── semantic_search.py        # Memory retrieval
│   ├── file_operations.py        # Sandboxed file I/O
│   └─ llm_client.py              # LM Studio integration
├─ logging/
│   ├── ritual_logger.py          # Consent-based audit trails
│   └─ living_scroll.py           # Session documentation
├─ sandbox/                       # Bounded file operations
├─ mem/                           # Agent scratch space
├─ out/                           # User artifacts
└─ logs/                          # Ritual logs & living scroll
```

Deployment Configuration

Local Model Requirements

- **Minimum:** 7B parameter model (Qwen2.5-7B-Instruct)
- **Recommended:** 14B+ parameter model for complex reasoning
- **Hardware:** 16GB+ VRAM for optimal performance, 32GB RAM recommended
- **Storage:** 50GB for models and working space

Environment Setup

```
# Install dependencies
pip install asyncio aiofiles openai pathlib dataclasses

# Configure LM Studio
# Download model: Qwen2.5-7B-Instruct-GGUF
# Start server: localhost:1234

# Initialize BrainBox directories
python -m brainbox.setup --create-directories --init-configs
```

Configuration Files

```
# config/brainbox.json
{
  "lm_studio": {
    "base_url": "http://localhost:1234/v1",
    "default_model": "qwen2.5-7b-instruct",
    "timeout": 30
  },
  "voices": {
    "max_parallel": 4,
    "timeout_seconds": 30,
    "intensity_scaling": true
  },
  "security": {
    "sandbox_root": "./sandbox",
    "max_file_size_mb": 10,
    "allowed_extensions": [".md", ".txt", ".json", ".py"]
  }
}
```

Testing Framework

Unit Tests

```
# tests/test_voice_agents.py
async def test_camera_obscura_scene_frame():
    agent = VoiceAgent("Camera Obscura")
    ctx = AgentContext("sunset over lake", "creative", 0.6)
    result = await agent.run_cycle(ctx)
    assert result.ok
    assert len(result.text) > 0
    assert "scene" in result.artifacts

# tests/test_kulawa_orchestration.py
async def test_intensity_voice_selection():
    kulawa = Kulawa()
    high_intensity_ctx = AgentContext("panic deadline", "overwhelm", 0.9)
    voices = kulawa.select_voices(high_intensity_ctx)
    assert "Dr. Ming" in voices
    assert "Doc Moss" in voices
    assert len(voices) >= 4 # High intensity = more voices
```

Integration Tests

```
# tests/test_orchestration_patterns.py
async def test_solo_with_support_pattern():
    result = await solo_with_support("Plan local-first architecture")
    assert "Plan:" in result["text"]
    assert "Risks:" in result["text"]
    assert result["artifacts"] is not None

async def test_unified_chorus_facebook():
    result = await unified_chorus_facebook("Album release announcement")
    assert len(result["text"]) <= 900 # Facebook post limit
    assert "artifacts" in result
    assert result["artifacts"]["path"].endswith(".md")
```

Performance Tests

```
# tests/test_performance.py
async def test_parallel_voice_execution():
    start = time.time()
    ctx = AgentContext("complex analysis task", "business", 0.8)
    result = await Kulawa().run(ctx)
    duration = time.time() - start
    assert duration < 60 # Should complete within 1 minute
    assert len(result["voices"]) >= 3
```

Production Considerations

Security Hardening

- **Sandbox Enforcement:** All file operations constrained to sandbox directory
- **Input Validation:** Schema validation for all agent inputs/outputs
- **Resource Limits:** Memory and CPU constraints per voice agent
- **Audit Logging:** Complete activity logs for security review

Performance Optimization

- **Connection Pooling:** Reuse LM Studio connections across requests
- **Async Execution:** Parallel voice processing with proper timeout handling
- **Caching:** Cache frequent semantic search results and model responses
- **Resource Management:** Queue management for high-load scenarios

Monitoring & Observability

- **Ritual Logs:** Structured logging for all agent activities
- **Performance Metrics:** Latency, success rates, resource usage
- **Error Tracking:** Comprehensive error reporting and recovery
- **Usage Analytics:** Voice selection patterns and user interaction data

Backup & Recovery

- **State Persistence:** All ritual logs and user artifacts backed up

- **Configuration Management:** Version-controlled configuration files
 - **Disaster Recovery:** Procedures for system restoration
 - **Data Export:** User data portability and migration tools
-

Migration Path

Phase 1: Core Framework (Week 1-2)

1. Implement voice agent framework with tool boundaries
2. Set up Kulawa orchestrator with basic voice selection
3. Create agent contracts and schema validation
4. Integrate AXIOM oversight and ritual logging

Phase 2: Tool Integration (Week 3-4)

1. Connect LM Studio client for model inference
2. Wire semantic search to existing memory index
3. Implement file operations with sandbox security
4. Test orchestration patterns end-to-end

Phase 3: Production Hardening (Week 5-6)

1. Add comprehensive error handling and recovery
2. Implement performance monitoring and optimization
3. Security audit and penetration testing
4. User documentation and deployment guides

Phase 4: Advanced Features (Week 7-8)

1. Add custom orchestration pattern creation
 2. Implement advanced voice capabilities
 3. Create user interface for pattern management
 4. Performance tuning and optimization
-

Success Metrics

Functional Requirements

- **Voice Execution:** 95%+ successful agent execution rate
- **Response Time:** <30 seconds for standard orchestration patterns
- **Tool Reliability:** 99%+ tool call success rate
- **Schema Compliance:** 100% schema validation pass rate

User Experience

- **Output Quality:** Coherent, useful responses from multi-agent orchestration
- **Error Recovery:** Graceful handling of agent failures
- **Consent-based processing** for complete audit trails and consent tracking
- **Documentation:** Complete audit trails in living scroll format

System Performance

- **Throughput:** Handle 10+ concurrent orchestration requests
 - **Resource Usage:** <8GB RAM, <50% CPU for typical workloads
 - **Reliability:** 99.9% uptime for local deployment
 - **Security:** Zero sandbox escapes or unauthorized file access
-

Conclusion

BrainBox v3.0 represents a complete therapeutic computing platform that transforms conversational AI into functional, ethical, and reliable agentic systems. The architecture provides clear separation of concerns, robust error handling, and consent-based operation while maintaining the emotional intelligence and therapeutic focus of the original voice family system.

The implementation framework is production-ready and requires only integration of existing LM Studio and memory index components to become fully operational. The system is designed for local deployment, ensuring user privacy and data sovereignty while providing enterprise-grade functionality and reliability.